

CONTENTS

Orden de Crecimiento	
Guía de estudio con tolerancia cero — Módulo 2, Programación 3 (FING UdelaR) — Kleinberg & Tardos, Cap. 2	
0. Por qué existe este tema	
1. Las cuatro (cinco) definiciones formales	
1.1 O grande — cota superior	
1.2 Ω grande — cota inferior	
1.3 Θ grande — cota ajustada (tight bound)	
1.4 o pequeña (little-o) — cota superior estricta	
1.5 ω pequeña (little-omega) — cota inferior estricta	
2. Cómo demostrar formalmente que $f(n)$ es $O(g(n))$	
3. Cómo refutar que $f(n)$ es $O(g(n))$	
4. Jerarquía de funciones comunes	
5. Propiedades algebraicas	
5.1 Transitividad	
5.2 Suma de funciones — domina el término de mayor orden	
5.3 Producto de funciones	
5.4 O y Ω son operaciones “inversas” entre sí	
5.5 Relación entre o pequeña y las demás	
6. Trampas típicas de examen	
Tabla resumen de las definiciones	
Ejercicios de repaso rápido (con respuesta)	

Resumen ultra-rápido (2 minutos)

.....

Orden de Crecimiento

GUÍA DE ESTUDIO CON TOLERANCIA CERO — MÓDULO 2, PROGRAMACIÓN 3 (FING UDELAR) — KLEINBERG & TARDOS, CAP. 2

0. Por qué existe este tema

Antes de poder comparar algoritmos hace falta un lenguaje preciso para decir “este algoritmo es más rápido que aquel” que no dependa de la computadora en la que se ejecuta, del compilador, ni de constantes de implementación. Contar el tiempo de ejecución exacto en segundos es inútil para razonar en general: la misma implementación corre distinto en dos máquinas. Lo que **sí** es una propiedad intrínseca del algoritmo es *cómo escala* su costo cuando el tamaño de la entrada n crece sin límite — esa es la idea de **orden de crecimiento** (o **notación asintótica**).

Kleinberg-Tardos motivan la definición de “tiempo polinómico” con una propiedad de escalado muy concreta: al duplicar el tamaño de la entrada, un algoritmo razonable debería enlentecerse a lo sumo por un **factor constante**. Esto excluye a los algoritmos de fuerza bruta (típicamente $\Theta(2^n)$ o peor), que se vuelven inmanejables mucho antes de llegar a instancias de tamaño moderado (ver la Tabla 2.1 de la diapositiva: con $n=1.000.000$, un algoritmo $O(n)$ tarda 1 segundo, pero uno $O(2^n)$ ya es intratable con $n=100$).

La notación O , Ω , Θ , o (γ y ω) formaliza esa idea de “cómo escala”, ignorando constantes multiplicativas y términos de orden inferior. Es, literalmente, la herramienta que se usa en **todos** los demás módulos del curso para justificar la complejidad de cualquier algoritmo — greedy, divide y vencerás, programación dinámica, flujos, NP-completitud. Por eso este módulo, aunque parezca “solo definiciones”, es la base analítica de todo lo que sigue.

Regla de oro que se repite en TODO el módulo: para probar una cota asintótica **siempre hay que exhibir las constantes explícitas** (c , o c_1 y c_2 , o n_0) que hacen cumplir la desigualdad — nunca alcanza con decir “se ve que...”. Y para **refutar** una cota, hay que probar que **ninguna** elección de esas constantes puede funcionar (por contradicción o tomando límite).

1. Las cuatro (cinco) definiciones formales

Todas las definiciones comparan una función $T(n)$ (típicamente el tiempo de ejecución de un algoritmo) contra una función de referencia $f(n)$. Se asume que $T(n)$ y $f(n)$ son funciones **no negativas** definidas sobre los naturales (o eventualmente no negativas — “asintóticamente no negativas”, como aclara la diapo 12).

1.1 O GRANDE — COTA SUPERIOR

$T(n)$ es $O(f(n))$ si $\exists c > 0, n_0 \geq 0$ tales que $\forall n \geq n_0: T(n) \leq c \cdot f(n)$.

Intuición: $T(n)$ no crece **más rápido** que $f(n)$, salvo por una constante multiplicativa, a partir de cierto punto. Es una cota *superior*, no necesariamente ajustada.

1.2 Ω GRANDE — COTA INFERIOR

$T(n)$ es $\Omega(f(n))$ si $\exists c > 0, n_0 \geq 0$ tales que $\forall n \geq n_0: T(n) \geq c \cdot f(n)$.

Intuición: $T(n)$ no crece **más lento** que $f(n)$ (salvo constante), a partir de cierto punto. Es exactamente la definición de O pero con la desigualdad invertida.

1.3 Θ GRANDE — COTA AJUSTADA (TIGHT BOUND)

$T(n)$ es $\Theta(f(n))$ si $T(n)$ es **$O(f(n))$ Y $\Omega(f(n))$** simultáneamente. Equivalentemente: $\exists c_1 > 0, c_2 > 0, n_0 \geq 0$ tales que $\forall n \geq n_0: c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$.

Intuición: $T(n)$ y $f(n)$ crecen exactamente al mismo ritmo, salvo constantes. Es la afirmación **más fuerte e informativa** de las tres — cuando se puede probar Θ , es preferible a solo O o solo Ω .

1.4 O PEQUEÑA (LITTLE-O) — COTA SUPERIOR ESTRICTA

$T(n)$ es $o(f(n))$ si $\forall \epsilon > 0$ (por más chico que sea) $\exists n_\epsilon \geq 0$ tal que $\forall n \geq n_\epsilon: T(n) < \epsilon \cdot f(n)$.

Esta es la definición que más diferencia genera con O . En O alcanza con **una** constante c fija, elegida una sola vez, que puede ser tan grande como haga falta ($c=1000000$ sirve igual que $c=2$). En o pequeña, la desigualdad tiene que valer para **cualquier** $\epsilon > 0$, sin importar cuán chico — esto fuerza a que el cociente $T(n)/f(n)$ tienda a 0, es decir, que $T(n)$ sea **estrictamente** de menor orden que $f(n)$, no solo “acotada por”.

1.5 Ω PEQUEÑA (LITTLE-OMEGA) — COTA INFERIOR ESTRICTA

$T(n)$ es $\omega(f(n))$ si $\forall \epsilon > 0 \exists n_\epsilon \geq 0$ tal que $\forall n \geq n_\epsilon: T(n) > \epsilon \cdot f(n)$.

Es el análogo estricto de Ω , del mismo modo que o es el análogo estricto de O . (Este módulo, siguiendo el material del curso, trabaja sobre todo con O , Ω , Θ y o ; ω pequeña aparece poco en los prácticos pero conviene saber su definición porque es simétrica y puede aparecer en el examen como comprobación de simetría de definiciones.)

Cuantificadores — el orden importa y es la fuente #1 de errores al escribir estas definiciones de memoria:

- O, Ω, Θ : $\exists$ constante(s) **fija(s)**, luego $\forall n$ suficientemente grande.
- o, ω : $\forall \epsilon$ (variable, arbitrariamente chico), y *para cada* ϵ existe un n_ϵ (que sí puede depender de ϵ) tal que $\forall n \geq n_\epsilon$.

Fijate que en o/ω el n_0 (acá llamado n_ϵ) **tiene permitido depender de ϵ** — de hecho, casi siempre crece cuando ϵ achica. Eso es lo que hace que la definición sea más exigente: no hay una única con-

stante que “salve” la desigualdad para siempre, tiene que funcionar para una familia infinita de exigencias cada vez más estrictas.

Forma equivalente por límite (diapo 15, útil como atajo de cálculo, no como definición formal a citar en un examen si piden “por definición”):

- Si $\lim_{n \rightarrow \infty} T(n)/f(n) = c$, con $0 < c < \infty$, entonces $T(n)$ es $\Theta(f(n))$.
- Si $\lim_{n \rightarrow \infty} T(n)/f(n) = 0$, entonces $T(n)$ es $o(f(n))$ (y por lo tanto también $O(f(n))$).
- Si $\lim_{n \rightarrow \infty} T(n)/f(n) = \infty$, entonces $T(n)$ es $\omega(f(n))$ (y por lo tanto también $\Omega(f(n))$).

Esto es exactamente la técnica usada en el Ejercicio 1 del Práctico 2 (funciones $f_1..f_5$): cuando la comparación directa es difícil, tomar logaritmo o cociente y evaluar el límite resultante.

2. Cómo demostrar formalmente que $f(n)$ es $O(g(n))$

Procedimiento general: hay que **exhibir** un par de constantes concretas (c, n_0) — no basta con argumentar en abstracto — y verificar que la desigualdad $f(n) \leq c \cdot g(n)$ se cumple para todo $n \geq n_0$. La técnica habitual es acotar cada término de $f(n)$ por separado usando que, para $n \geq 1$, las potencias de n son crecientes ($n \leq n^2 \leq n^3 \leq \dots$), y agrupar.

Ejemplo 2.1 — $T(n) = 32n^2 + 17n + 1$ es $O(n^2)$ (ejemplo de la diapo 11).

Objetivo: encontrar c, n_0 tales que $32n^2 + 17n + 1 \leq c \cdot n^2$ para todo $n \geq n_0$.

Truco estándar: para $n \geq 1$, cada término de menor grado se puede acotar por el término de mayor grado: $17n \leq 17n^2$ y $1 \leq n^2$ (ambas válidas para $n \geq 1$). Entonces:

$32n^2 + 17n + 1 \leq 32n^2 + 17n^2 + n^2 = 50n^2$ para todo $n \geq 1$.

Tomamos **$c = 50, n_0 = 1$** .

Verificación numérica (evaluando la desigualdad con valores concretos de n):

n	$T(n)=32n^2+17n+1$	$50n^2$	$\zeta T(n) \leq 50n^2?$
1	50	50	Sí (igualdad exacta)
2	163	200	Sí
5	886	1250	Sí
10	3371	5000	Sí
100	321701	500000	Sí

La desigualdad se cumple en $n=1$ con igualdad exacta ($50=50$) y se vuelve cada vez más holgada para n mayores, como se espera de una cota superior no ajustada. Chequeado computacionalmente para todo n entre 1 y 200: se cumple en el 100% de los casos. ■

Ejemplo 2.2 — $f(n) = 3n^2 + 5n + 2$ es $O(n^2)$, con constantes distintas a las del ejemplo anterior (para mostrar que c y n_0 NO son únicos).

Intento con n_0 grande y c chica: probamos $c=4$. Necesitamos $3n^2+5n+2 \leq 4n^2$, es decir $5n+2 \leq n^2$, es decir $n^2 - 5n - 2 \geq 0$. Resolviendo la cuadrática asociada $n^2 - 5n - 2 = 0$, la raíz positiva es $n \approx 5.37$, así que para $n \geq 6$ (entero) la desigualdad ya vale con margen.

Tomamos **$c = 4$, $n_0 = 6$** .

Verificación numérica:

n	$f(n)=3n^2+5n+2$	$4n^2$	$\text{¿}f(n) \leq 4n^2\text{?}$
4	70	64	No (n_0 aún no alcanzado)
5	102	100	No (n_0 aún no alcanzado)
6	140	144	Sí — primer n donde se cumple
7	184	196	Sí
10	352	400	Sí

Chequeado computacionalmente que, en efecto, para $n=4$ y $n=5$ la desigualdad falla ($70 > 64$ y $102 > 100$), y que se cumple para todo n desde 6 hasta 200 sin excepción. Esto confirma que $n_0=6$ es necesario con esta elección de $c=4$ (con $c=50$ como en el Ejemplo 2.1, $n_0=1$ ya alcanzaba) — **el par (c, n_0) no es único**, hay infinitas combinaciones válidas, y el examen acepta cualquiera que se verifique correctamente. ■

Método general para $f(n) = a_d n^d + \dots + a_1 n + a_0$ con $a_d > 0$ (caso polinomios, diapo 16): siempre es $\Theta(n^d)$. La demostración de la cota O es exactamente el patrón de arriba: acotar cada término por el de grado d usando $n \geq 1$, y sumar los $|a_i|$ como constante c . La cota Ω sale de quedarse solo con el término dominante y acotar el resto por abajo (ver Sección 5).

3. Cómo refutar que $f(n)$ es $O(g(n))$

Para probar que $f(n)$ **no** es $O(g(n))$ hay que mostrar que **ninguna** elección de c y n_0 puede hacer cumplir $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. Hay dos técnicas estándar:

Técnica A — por contradicción: se asume que sí existen c, n_0 que cumplen la desigualdad, y se deriva un absurdo (una desigualdad que es falsa para n suficientemente grande, sin importar cuál sea c).

Técnica B — por límite: si $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$ (o no existe por oscilación sin estar acotado), entonces $f(n)$ no puede ser $O(g(n))$, porque O exige que ese cociente quede acotado por una constante c eventualmente.

Ejemplo 3.1 — Refutar que $2n^2$ es $O(n)$.

Por contradicción: supongamos que $2n^2$ es $O(n)$. Entonces existirían $c > 0$, $n_0 \geq 0$ tales que $2n^2 \leq c \cdot n$ para todo $n \geq n_0$. Como $n > 0$ para $n \geq n_0 \geq 1$, podemos dividir ambos lados por n sin invertir la desigualdad: $2n \leq c$ para todo $n \geq n_0$. Pero esto dice que **$2n$ está acotada por una constante c** para todo n suficientemente grande — absurdo, porque $2n \rightarrow \infty$ cuando $n \rightarrow \infty$ (para cualquier c fija, basta tomar $n > c/2$ para que $2n$ supere a c). Contradicción. Por lo tanto $2n^2$ no es $O(n)$. ■

Verificación numérica de la contradicción (mostrando que para cualquier c , siempre hay un n donde falla):

c elegida	n donde $2n$ supera a c	$2n$ en ese punto
10	6	$12 > 10$
100	51	$102 > 100$
1000	501	$1002 > 1000$
10000	5001	$10002 > 10000$

No importa cuán grande se elija c , siempre existe un n (concretamente, $n = \lceil c/2 \rceil + 1$) que rompe la desigualdad $2n \leq c$. Esto confirma computacionalmente que ninguna constante c “salva” la cota. ■

Ejemplo 3.2 — Refutar por límite: $n!$ no es $O(2^n)$.

$\lim_{n \rightarrow \infty} n!/2^n = \infty$ (el factorial crece más rápido que cualquier exponencial de base fija, porque el i -ésimo factor de $n!$ es i , que eventualmente supera y sigue creciendo más que la base fija 2 del denominador en cada paso adicional del cociente). Como el límite del cociente diverge a infinito en vez de quedar acotado, no puede existir una constante c tal que $n! \leq c \cdot 2^n$ eventualmente. Por lo tanto $n!$ no es $O(2^n)$. (De hecho, $n!$ es $\omega(2^n)$, la cota estricta inferior.) ■

⚠ Error frecuente de tolerancia cero: “probar” que $f(n)$ no es $O(g(n))$ mostrando un solo n donde $f(n) > c \cdot g(n)$ para una c particular elegida por el estudiante. Esto NO refuta nada: la definición de O dice “existe alguna c ”, así que hay que descartar **todas** las c posibles, no solo una. La forma correcta es la de arriba: suponer que existe una c genérica y llegar a una contradicción que valga para cualquier c , o usar el argumento de límite.

4. Jerarquía de funciones comunes

Orden de crecimiento ascendente (cada función es o-pequeña de la siguiente, es decir, estrictamente más lenta):

$$1 \text{ (constante)} < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < n^k \text{ (k fijo)} < 2^n < 3^n < \dots < n! < n^n$$

(donde “<” se lee “es o-pequeña de”, es decir, estrictamente más lenta que).

Ejemplo 4.1 — Verificación numérica de la jerarquía $\log n < n < n \log n < n^2$.

n	$\log_2(n)$	n	$n \cdot \log_2(n)$	n^2
10	3.32	10	33.22	100
100	6.64	100	664.39	10.000
1.000	9.97	1.000	9.965.78	1.000.000
10.000	13.29	10.000	132.877	100.000.000

Se ve numéricamente cómo, a medida que n crece, la brecha entre columnas consecutivas se ensancha cada vez más rápido — eso es justamente lo que dice $\log n = o(n)$: el cociente $\log(n)/n \rightarrow 0$.

Reglas de la diapositiva 16 (Kleinberg-Tardos), útiles para no tener que probar todo desde cero:

- **Polinomios:** $T(n) = a_0 + a_1n + \dots + a_d \cdot n^d$ con $a_d > 0 \Rightarrow T(n)$ es $\Theta(n^d)$. (El término dominante determina el orden — ver Sección 5.)
- **Logaritmos, cualquier base:** $\Theta(\log_a n) = \Theta(\log_b n)$ para cualquier par de constantes $a, b > 0$. Esto es porque $\log_a(n) = \log_b(n)/\log_b(a)$, y $1/\log_b(a)$ es una constante — cambiar de base solo multiplica por una constante, lo cual no afecta la notación Θ . **Por eso en notación asintótica nunca se especifica la base del logaritmo** (salvo que el exponente de otra función dependa de ella, lo cual no es el caso típico de este curso).
- **Logaritmos vs. polinomios:** para todo $d > 0$, $\log n$ es $O(n^d)$ — el logaritmo, aunque tiende a infinito, es asintóticamente más chico que **cualquier** potencia positiva de n , por más chica que sea (incluso $n^{0.001}$).
- **Polinomios vs. exponenciales:** para todo $r > 1$ y todo $d > 0$, n^d es $O(r^n)$ — cualquier polinomio, sin importar su grado, es asintóticamente más chico que cualquier exponencial de base $r > 1$ fija.

Ejemplo 4.2 — Práctico 2, Ej.1: comparar $2^{\sqrt{\log_2 n}}$ con las familias básicas.

Sea $z = \log_2(n)$. Las funciones dadas se reescriben como funciones de z : $\log(f_2(n)) = z/3$, $\log(f_4(n)) = \log_2 z$, $\log(f_5(n)) = z^{1/2}$ (donde $f_2(n) = n^{1/3}$, $f_4(n) = \log_2 n$, $f_5(n) = 2^{\sqrt{\log_2 n}}$).

Comparando $\log_2 z$ vs. $z^{1/2}$: para $z \geq 16$, $\log_2 z \leq z^{1/2}$ (verificable: en $z=16$, $\log_2 16=4$ y $16^{0.5}=4$, igualdad; para $z > 16$ la raíz cuadrada crece más rápido). Como $z = \log_2 n$, la condición $z \geq 16$ equivale a $n \geq 2^{16} = 65.536$.

Comparando $z^{1/2}$ vs. $z/3$: para $z \geq 9$, $z^{1/2} \leq z/3$ (en $z=9$: $3 \leq 3$, igualdad; para $z > 9$ el término lineal gana). Equivale a $n \geq 2^9 = 512$.

Conclusión: $f_5(n) = 2^{\sqrt{\log_2 n}}$ queda intercalada estrictamente entre $f_4(n) = \log_2 n$ y $f_2(n) = n^{1/3}$ en la jerarquía — es un ejemplo clásico de función con crecimiento “entre logarítmico y polinomial”, útil para entender que la jerarquía de arriba tiene infinitos escalones intermedios, no son las únicas funciones que existen.

Truco práctico para comparar dos funciones “raras”: tomar logaritmo de ambas (si son crecientes y positivas, el logaritmo preserva el orden salvo en casos degenerados — ver Ejercicio 1(c)/(d) del Práctico 2 sobre esa hipótesis extra) y comparar las versiones “logueadas”, que suelen ser más simples.

5. Propiedades algebraicas

5.1 TRANSITIVIDAD

Si $f(n)$ es $O(g(n))$ y $g(n)$ es $O(h(n))$, entonces $f(n)$ es $O(h(n))$. (Análogamente para Ω y Θ .) Esto permite encadenar comparaciones sin volver a las definiciones cada vez — por ejemplo, para probar $f(n)=O(n^3)$ alcanza con probar $f(n)=O(n^2)$ y usar que n^2 es $O(n^3)$.

5.2 SUMA DE FUNCIONES — DOMINA EL TÉRMINO DE MAYOR ORDEN

Si $f(n)$ es $O(g(n))$ y $h(n)$ es $O(g(n))$, entonces $f(n)+h(n)$ es $O(g(n))$ (la suma de dos funciones $O(g)$ sigue siendo $O(g)$, tomando como c la suma de las constantes de cada una). Más en general, si $T(n) = f_1(n) + f_2(n) + \dots + f_k(n)$ con k constante, entonces $T(n)$ es $\Theta(\max\{f_1(n), \dots, f_k(n)\})$ — el término de mayor orden **absorbe** a los demás en la notación asintótica.

Ejemplo 5.1 — Verificación de “domina el término mayor”.

$T(n) = n^3 + 100n^2 + 1000n + 5000$. Afirmando $T(n)$ es $\Theta(n^3)$.

Cota O: para $n \geq 1$, $100n^2 \leq 100n^3$, $1000n \leq 1000n^3$, $5000 \leq 5000n^3$, entonces $T(n) \leq (1+100+1000+5000)n^3 = 6101n^3$. Tomamos $c_2=6101$, $n_0=1$.

Cota Ω : para $n \geq 1$, $T(n) \geq n^3$ (todos los demás términos son positivos, así que $T(n) \geq n^3$ trivialmente). Tomamos $c_1=1$, $n_0=1$.

Con $c_1=1$, $c_2=6101$, $n_0=1$: $T(n)$ es $\Theta(n^3)$. **Verificación numérica en $n=1000$:**

$T(1000) = 1.000.000.000 + 100.000.000 + 1.000.000 + 5.000 = 1.101.005.000$. Y $n^3 = 10^9 = 1.000.000.000$. Efectivamente $1 \cdot 10^9 \leq 1.101.005.000 \leq 6101 \cdot 10^9 \checkmark$.

5.3 PRODUCTO DE FUNCIONES

Si $f_1(n)$ es $O(g_1(n))$ y $f_2(n)$ es $O(g_2(n))$, entonces $f_1(n) \cdot f_2(n)$ es $O(g_1(n) \cdot g_2(n))$ (se toma $c = c_1 \cdot c_2$ y $n_0 = \max(n_{01}, n_{02})$). Análogamente para Ω y Θ .

⚠ Error frecuente de tolerancia cero — no todas las operaciones preservan O de la misma manera (Ejercicio 1 del Práctico 2, Kleinberg-Tardos Ex. 2.5):

Dado que $f(n)$ es $O(g(n))$:

- **$f(n)^2$ es $O(g(n)^2)$** — VERDADERO (elevar al cuadrado una desigualdad entre cantidades positivas la preserva).

- **$2^{f(n)}$ es $O(2^{g(n)})$** — FALSO en general. Contraejemplo verificado: $f(n)=2n$, $g(n)=n$ (cumple $f=O(g)$ con $c=2$). Pero $2^{f(n)}=2^{(2n)}=4^n$ y $2^{g(n)}=2^n$. Si 4^n fuera $O(2^n)$, existiría c tal que $4^n \leq c \cdot 2^n \Leftrightarrow$

$2^n \leq c$ para todo n grande — imposible porque $2^n \rightarrow \infty$ (mismo argumento del Ejemplo 3.1).

Exponenciar no preserva O.

• **$\log(f(n))$ es $O(\log(g(n)))$** — verdadero **solo con una hipótesis extra** (que $g(n) > 1 + \varepsilon$ eventualmente, para algún $\varepsilon > 0$ fijo). Sin esa hipótesis es falso: contraejemplo $f(n)=2$, $g(n)=1$ (constantes; cumple $f=O(g)$ con $c=2$). Pero $\log(f(n))=\log(2)>0$ para todo n , mientras $\log(g(n))=\log(1)=0$ para todo n , y $\log(2)$ no puede ser $O(0)$. **Tomar logaritmo tampoco preserva O sin condiciones adicionales** — la clave es que necesita un “piso” para que $\log(g(n))$ no se pegue a 0 o sea negativo.

La moraleja de examen: **cuadrado sí preserva O siempre; exponencial y logaritmo NO preservan O automáticamente** — hay que revisar las hipótesis con cuidado antes de aplicar la operación a ambos lados de una relación $O/\Omega/\Theta$.

5.4 O Y Ω SON OPERACIONES “INVERSAS” ENTRE SÍ

Proposición (Ejercicio 2 del Práctico 2): si $f(n)$ es $O(g(n))$, entonces $g(n)$ es $\Omega(f(n))$.

Demostración: de $f(n)$ es $O(g(n))$ sale que existen c, n_0 tales que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$. Como $c > 0$, dividimos ambos lados por c sin invertir la desigualdad: $g(n) \geq (1/c) \cdot f(n)$ para todo $n \geq n_0$. Tomando la constante $1/c$ (que también es > 0) y el mismo n_0 , esto es exactamente la definición de $g(n)$ es $\Omega(f(n))$. ■

5.5 RELACIÓN ENTRE O PEQUEÑA Y LAS DEMÁS

- $o(f) \Rightarrow O(f)$, pero $O(f)$ no implica $o(f)$ en general (o es estrictamente más fuerte). Prueba: la definición de o exige la desigualdad para *todo* $\varepsilon > 0$; en particular vale para $\varepsilon=1$, lo que da exactamente la definición de O con $c=1$.
- $o(f) \Rightarrow \lim T(n)/f(n) = 0$ (consecuencia directa de reescribir la definición de límite).
- $o(f)$ excluye ser $\Omega(f)$ (y por lo tanto excluye ser $\Theta(f)$): si $T(n)$ fuera simultáneamente $o(f(n))$ y $\Omega(f(n))$, tomando $\varepsilon=c$ (la constante de la definición de Ω) se llega a $T(n) \geq c \cdot f(n)$ y $T(n) < c \cdot f(n)$ a la vez para n grande — contradicción directa.

Contraejemplo de referencia para no confundir O con o (Práctico 2, Ej.3.c): $f(n)=2n$ es $O(n)$ (con $c=2$), pero **no** es $o(n)$: tomando $\varepsilon=1$, se necesitaría $2n < n$ eventualmente, es decir $n < 0$ — absurdo. La regla práctica: si $f(n)=\Theta(g(n))$ con un cociente que tiende a una constante positiva (no a 0), entonces $f(n)$ nunca es $o(g(n))$ — o exige que el cociente se vaya estrictamente a cero.

6. Trampas típicas de examen

⚠ Trampa 1 — confundir “cota superior” (O) con “cota ajustada” (Θ).

Decir “el algoritmo es $O(n^2)$ ” es una afirmación **débil**: solo dice que el algoritmo no es peor que n^2 , pero podría en realidad ser mucho más rápido (por ejemplo, $O(n)$ también sería cierto si el algoritmo es realmente lineal, ya que $O(n) \Rightarrow O(n^2)$ por transitividad hacia arriba... pero **no al revés**). Cuando el enunciado pide “demostrar la complejidad” de un algoritmo, casi siempre se espera una cota **Θ ajustada** (cota superior Y cota inferior), no solo O . La diapo 13 lo marca explícitamente como

“afirmación sin sentido”: decir que un algoritmo “requiere al menos $O(n \log n)$ comparaciones” mezcla mal los conceptos — una cota *inferior* (“requiere al menos”) se expresa con Ω , nunca con O .

⚠ Trampa 2 — comparar polinomios de distinto grado sin cuidado con signos o coeficientes negativos.

$f(n)$ es $O(g(n))$ compara el comportamiento **asintótico**, no el valor en n chico. Un polinomio de grado 2 con coeficiente líder gigante (ej. $10^9 n^2$) sigue siendo $O(n^3)$ y sigue siendo **peor** asintóticamente que un polinomio de grado 3 con coeficiente líder minúsculo (ej. $0.0001 n^3$) — para n suficientemente grande, el de grado mayor termina superando al de grado menor sin importar las constantes. El error típico es “mirar los números” en vez de mirar el exponente dominante.

⚠ Trampa 3 — escribir la base del logaritmo como si importara en notación asintótica.

Como se vio en 4, $\log_a(n)$ y $\log_b(n)$ difieren solo por una constante multiplicativa (el factor $1/\log_b(a)$), así que $\Theta(\log_2 n) = \Theta(\log_{10} n) = \Theta(\ln n)$. **Nunca hace falta (ni tiene sentido) especificar la base dentro de la notación $O/\Omega/\Theta/o$** , salvo que la base misma dependa de n (caso raro, fuera del alcance habitual del curso). Escribir “ $O(\log_2 n) \neq O(\log_{10} n)$ ” como si fueran cotas distintas es un error de tolerancia cero.

⚠ Trampa 4 — probar una cota Ω “en general” sin construir una instancia explícita.

Para probar que el **peor caso** de un algoritmo es $\Omega(f(n))$, no alcanza con decir “puede pasar que tarde esto” de manera vaga. Hay que **construir una familia de instancias concreta, parametrizada por n** , y demostrar que sobre esa familia específica el algoritmo ejecuta al menos $c \cdot f(n)$ pasos. El ejemplo de referencia del curso es la cota $\Omega(n^2)$ para Gale-Shapley (Ejercicio 2, Práctico 2): se construye explícitamente una instancia con listas de preferencia opuestas entre hombres y mujeres que fuerza $\sum_{i=1}^n (n-i+1) = n(n+1)/2 = \Omega(n^2)$ propuestas totales, y se argumenta que **toda implementación correcta** del algoritmo, al ser G-S determinista en su comportamiento observable sobre esa instancia, realiza esa misma cantidad de trabajo — independientemente de detalles como el orden de recorrido de los hombres libres.

⚠ Trampa 5 — el signo “=” en notación O es un abuso de notación, no una igualdad real.

La diapo 12 lo marca explícitamente: $O(f(n))$ es en rigor un **conjunto de funciones** (todas las que cumplen la cota), y escribir “ $T(n) = O(f(n))$ ” es abuso de notación por “ $T(n) \in O(f(n))$ ”. El error de tolerancia cero es usar la transitividad del signo “=” para deducir disparates: de $f(n)=O(n^3)$ y $g(n)=O(n^3)$ NO se sigue $f(n)=g(n)$ (contraejemplo trivial: $f(n)=5n^3$, $g(n)=3n^3$; ambas son $O(n^3)$, pero claramente $f(n) \neq g(n)$). “Está bien abusar de la notación, pero no está bien usarla mal” (cita textual de la diapositiva del curso).

Nota sobre alcance del módulo: el **Teorema Maestro** para resolver recurrencias de divide y vencerás ($T(n) = a \cdot T(n/b) + f(n)$) **no forma parte de este módulo**. Ese contenido corresponde al Módulo 5 (Divide y Vencerás), cuando se empiezan a analizar algoritmos recursivos que dividen el problema en

subproblemas. En este módulo el análisis asintótico se aplica solo a funciones de costo ya calculadas explícitamente (sumas, productos, comparaciones directas), no a recurrencias.

Tabla resumen de las definiciones

Notación	Se lee	Cuantificadores	Desigualdad	Rol
$O(f(n))$	"es a lo sumo del orden de f "	$\exists c > 0, n_0 \geq 0, \forall n \geq n_0$	$T(n) \leq c \cdot f(n)$	Cota superior (no necesariamente ajustada)
$\Omega(f(n))$	"es al menos del orden de f "	$\exists c > 0, n_0 \geq 0, \forall n \geq n_0$	$T(n) \geq c \cdot f(n)$	Cota inferior (no necesariamente ajustada)
$\Theta(f(n))$	"es exactamente del orden de f "	$\exists c_1, c_2 > 0, n_0 \geq 0, \forall n \geq n_0$	$c_1 f(n) \leq T(n) \leq c_2 f(n)$	Cota ajustada (O y Ω a la vez)
$o(f(n))$	"es estrictamente más chico que f "	$\forall \varepsilon > 0, \exists n_\varepsilon \geq 0, \forall n \geq n_\varepsilon$	$T(n) < \varepsilon \cdot f(n)$	Cota superior estricta (cociente $\rightarrow 0$)
$\omega(f(n))$	"es estrictamente más grande que f "	$\forall \varepsilon > 0, \exists n_\varepsilon \geq 0, \forall n \geq n_\varepsilon$	$T(n) > \varepsilon \cdot f(n)$	Cota inferior estricta (cociente $\rightarrow \infty$)

Ejercicios de repaso rápido (con respuesta)

#	Enunciado	Respuesta
1	Escribí la definición formal de “ $T(n)$ es $O(f(n))$ ” con cuantificadores.	$\exists c > 0, n_0 \geq 0: \forall n \geq n_0, T(n) \leq c \cdot f(n)$
2	¿Cuál es la diferencia clave entre O y o pequeña?	O : alcanza con UNA constante c fija. o : debe valer para CUALQUIER $\varepsilon > 0$ (cociente $\rightarrow 0$)
3	$f(n) = 5n^2 + 3n$ es Θ (¿qué?)	$\Theta(n^2)$ — el término dominante determina el orden
4	¿ $o(f)$ implica $O(f)$? ¿Y al revés?	Sí implica $O(f)$ (tomando $\varepsilon = 1$). El revés NO vale en general (contraejemplo: $f(n) = 2n$ es $O(n)$ pero no $o(n)$)
5	¿Si $f(n)$ es $O(g(n))$, qué se puede decir de $g(n)$?	$g(n)$ es $\Omega(f(n))$ — son “inversas” (dividiendo la desigualdad por c)
6	¿ $2^{f(n)}$ es $O(2^{g(n)})$ si $f(n)$ es $O(g(n))$?	FALSO en general (contraejemplo: $f(n) = 2n$, $g(n) = n \Rightarrow 4^n$ no es $O(2^n)$)
7	¿ $f(n)^2$ es $O(g(n)^2)$ si $f(n)$ es $O(g(n))$?	VERDADERO (elevar al cuadrado preserva la desigualdad entre positivos)
8	¿Por qué nunca se especifica la base de un logaritmo en notación O ?	Porque cambiar de base solo multiplica por una constante ($\log_a n = \log_b n / \log_b a$), lo cual no afecta la notación asintótica
9	Ordená: n , $\log n$, n^2 , $n \log n$, 2^n , $n!$	$\log n < n < n \log n < n^2 < 2^n < n!$
10	¿Cómo se prueba una cota Ω ajustada para el peor caso de un algoritmo?	Construyendo una instancia explícita, parametrizada por n , sobre la cual el algoritmo hace $\geq c \cdot f(n)$ pasos — no alcanza con argumentar en general
11	¿“ $T(n) = O(f(n))$ ” es una igualdad real?	No — es abuso de notación por $T(n) \in O(f(n))$; $O(f(n))$ es un conjunto de funciones
12	¿Dónde se estudia el Teorema Maestro en este curso?	En el Módulo 5 (Divide y Vencerás), no en este módulo

Resumen ultra-rápido (2 minutos)

- **$O(f)$** : $\exists c, n_0: T(n) \leq c \cdot f(n) \forall n \geq n_0$ — cota superior, no necesariamente ajustada.
- **$\Omega(f)$** : $\exists c, n_0: T(n) \geq c \cdot f(n) \forall n \geq n_0$ — cota inferior, no necesariamente ajustada.
- **$\Theta(f) = O(f) \cap \Omega(f)$** : cota ajustada — la afirmación más fuerte, preferible cuando se puede probar.
- **$o(f)$** : $\forall \varepsilon > 0 \exists n_\varepsilon: T(n) < \varepsilon \cdot f(n) \forall n \geq n_\varepsilon$ — cota superior estricta (cociente $\rightarrow 0$), más fuerte que O .
- **$\omega(f)$** : análogo estricto de Ω (cociente $\rightarrow \infty$).

- **Para probar $O/\Omega/\Theta$:** exhibir constantes explícitas y verificar la desigualdad — nunca alcanza con argumentar vagamente.
- **Para refutar O :** por contradicción (asumir que existe c genérica y llegar a un absurdo) o por límite (mostrar que el cociente diverge).
- **Jerarquía:** $1 < \log n < n < n \log n < n^2 < n^3 < \dots < 2^n < n! < n^n$.
- **Suma:** domina el término de mayor orden. **Producto:** se multiplican las cotas. **Transitividad:** encadenable.
- **Cuadrado preserva O ; exponencial y logaritmo NO preservan O sin hipótesis extra.**
- **Peor caso Ω :** requiere construir una instancia explícita, no alcanza con argumentar en general.
- **El signo “=” en notación O es abuso de notación** — $O(f)$ es un conjunto, no un valor.
- **Nunca especificar base de logaritmo** en notación asintótica (todas son equivalentes salvo constante).
- **Teorema Maestro:** fuera de este módulo, corresponde al Módulo 5 (D&C).